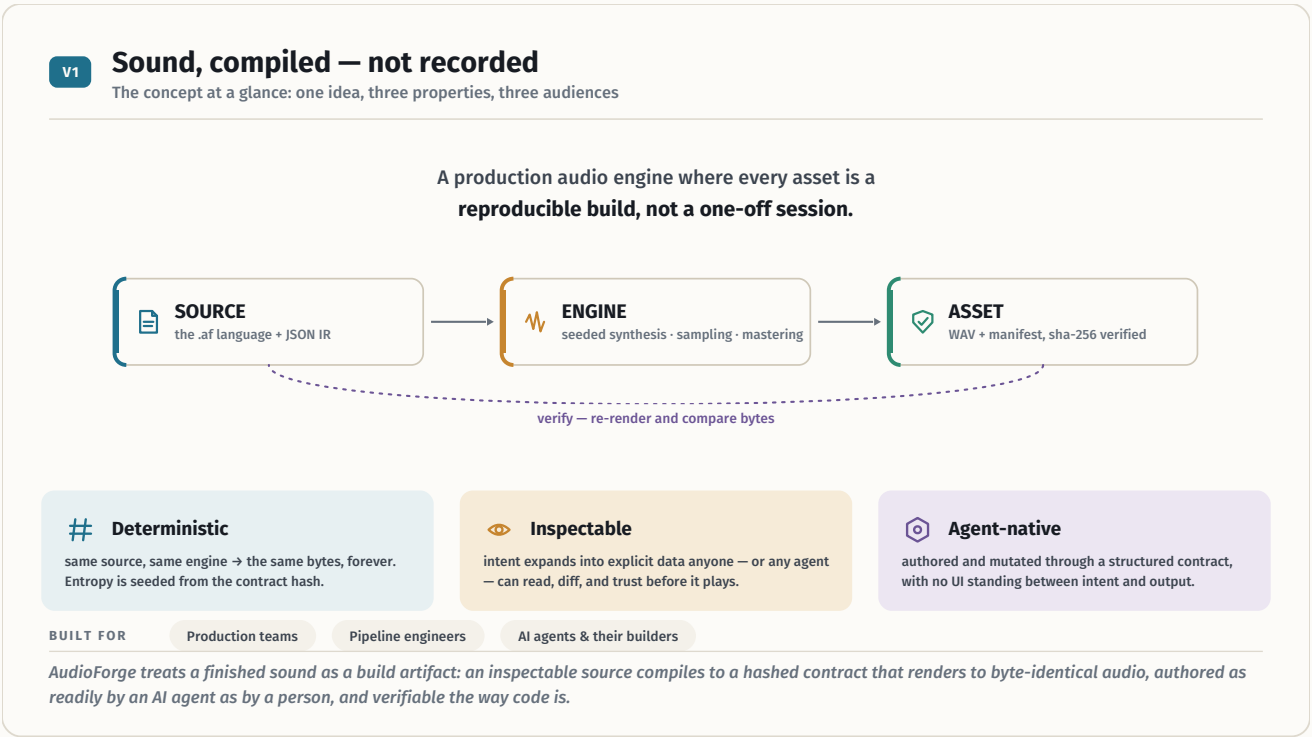
CONCEPT PAPER

Non-vocal audio · deterministic · agent-native

AudioForge

Sound, compiled — not recorded.

A production audio engine in which every asset is a build artifact — reproducible byte-for-byte from an inspectable contract, and authored as readily by an AI agent as by a person.



PIPELINE
.af → IR → WAV + manifest

GUARANTEE
same source + engine → same bytes

FOR
teams · engineers · AI agents

A crystallized concept — problem, model, capabilities, experience, principles, direction.

1 SECTION Core concept

AudioForge is a production audio engine in which sound is compiled, not recorded: every asset is a build artifact, reproducible byte-for-byte from an inspectable contract, and authored as readily by an AI agent as by a person.

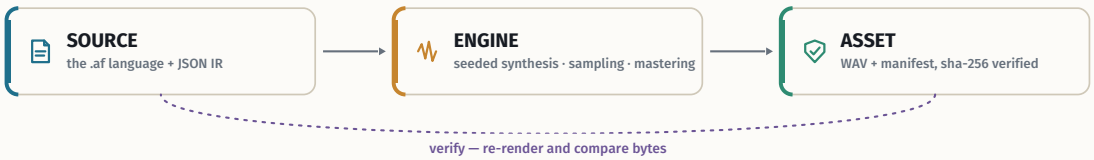
Most generated audio arrives as an opaque file — the end of a session that lived in someone's memory and one particular arrangement of tools. AudioForge replaces that with a pipeline that behaves like a compiler. A concise source language states intent; it compiles to a JSON intermediate representation — the IR — that is the single, explicit ground truth for what will be produced. The engine renders that IR to a WAV file and a manifest recording the source's content hash and the output's checksum. Given the same source and the same engine, the same bytes come out every time.

The move is small to state and large in consequence. When the artifact is a build, everything that follows from build systems becomes available to sound: diffs, review, provenance, caching, regression gates, and machine authorship. Creative intent stays first-class — but intent must become explicit data before it can play, so nothing that affects the result stays hidden. The scope is deliberately non-vocal: music, sound design, effects, ambience, loops, cues, and sonic identity, rather than voice or lyrics.

V1 Sound, compiled — not recorded

The concept at a glance: one idea, three properties, three audiences

A production audio engine where every asset is a **reproducible build, not a one-off session.**



Deterministic
same source, same engine → the same bytes, forever. Entropy is seeded from the contract hash.

Inspectable
intent expands into explicit data anyone — or any agent — can read, diff, and trust before it plays.

Agent-native
authored and mutated through a structured contract, with no UI standing between intent and output.

BUILT FOR Production teams · Pipeline engineers · AI agents & their builders

AudioForge treats a finished sound as a build artifact: an inspectable source compiles to a hashed contract that renders to byte-identical audio, authored as readily by an AI agent as by a person, and verifiable the way code is.

2 SECTION The problem

Production audio is trusted to a process that cannot keep its promises. A finished sound is the residue of a manual session — plugin states, tool versions, hand-tuned moves, and a good deal of memory. Reproducing it later is a negotiation, not a guarantee. When an asset must change, the team cannot answer the questions that matter: what changed, why, and whether the new version still matches the original intent.

Two forces make this worse. First, randomness leaks. Noise, reverb tails, dither, and plugin entropy are typically drawn from the wall clock or a global generator, so “the same project” produces different bytes on different runs. Variation that should be a deliberate, traceable choice arrives instead as drift no one asked for. Second, the producer is increasingly not a person. AI agents now generate content directly, but they have no structured, inspectable, verifiable surface for audio — they are left driving interfaces built for humans or emitting blobs no pipeline can check.

That last point compounds the rest. Models of this class produce errors as a statistical property, not an exception: omissions, misreadings, silent partial failures. A pipeline that consumes model output without verifying it has not made a small oversight; it has left out a load-bearing layer. The problem is urgent now because the volume of generated audio, the number of non-human producers, and the demand for provenance are all rising at once.

V2

From broken promise to build artifact

Each failure of today's workflow, and the mechanism that resolves it into value

THE PROBLEM	THE MECHANISM	THE VALUE
Sessions no one can repeat; hidden plugin and tool state.	The IR is truth — intent compiles to explicit, diffable data.	Every render reproducible and reviewable from its source.
Randomness leaks from clocks and global RNG.	All entropy is seeded from the contract hash.	Declared, traceable variation instead of accidental drift.
"What changed, and does it still match intent?" goes unanswered.	A manifest records the spec hash and output checksum.	Diffable provenance; drift caught by a byte-compare gate.
Agents emit opaque blobs no pipeline can check.	Validation binds resources: valid means render-ready.	Agent output is verified before it enters production.

The through-line: opacity becomes an inspectable contract, leaked randomness becomes seeded lineage, and unverifiable output becomes a gate — turning fragile sessions into trustworthy deliveries.

3 SECTION Intended users

AudioForge serves three audiences whose needs converge on the same properties from different directions.

Production teams

People responsible for delivering audio assets at volume and standing behind them. They need outputs they can reproduce on demand, stems they can re-mix, manifests they can audit, and a way to prove a delivery still matches what was approved. Their enemy is silent change.

Developers and pipeline engineers

People who wire audio into build systems, content pipelines, and games. They want sound to behave like every other build output: deterministic, cacheable, diffable, and guarded by tests. Their enemy is non-determinism that cannot be gated.

AI agents and their builders

Autonomous producers and the teams who deploy them. They need a surface they can read and write directly — a contract with explicit fields, structured diagnostics, and a validation step that means "this will render." Their enemy is ambiguity with no human present to absorb it.

4 SECTION Value proposition

The same guarantee pays out differently to each audience.

For production teams	Trust becomes mechanical. Every delivery carries a hash of its source and a checksum of its output; two renders compare byte-for-byte; variants and cutdowns inherit a documented lineage. “Reproduce the approved version” is a command, not an archaeology project.
For engineers	Audio joins the build. A verify step re-renders from source and compares bytes, turning drift into a failing check rather than a late surprise. Determinism holds within an environment and is proven across environments by that same gate, so audio can live in CI beside code.
For agents	The surface is safe to author against. The contract is explicit and diffable; validation binds real resources and returns a true render-ready predicate; every problem comes back as structured, line-cited feedback rather than a crash. An agent can propose, inspect, and verify its own output before anything reaches production.

> *The common thread: the file stops being the source of truth. The contract is.*

5 SECTION Operating model

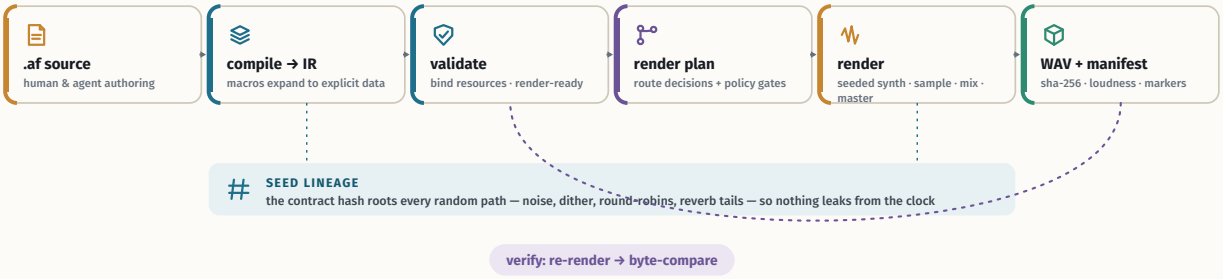
The system runs as a chain of explicit stages, each producing an artifact the next stage — and any reviewer — can inspect.

1. **Author**
   a concise language expresses intent for people and agents; high-level ideas are permitted, but only as sugar.
2. **Compile**
   sugar expands into explicit parameters and the IR is canonically hashed, with the engine version folded in, so any change that could alter the bytes changes the identity.
3. **Validate**
   beyond a shape check, validation binds every resource the render will touch; a pass is a true statement that the asset will render, and problems return as line-cited diagnostics.
4. **Plan**
   a render plan records each routing decision, the evidence behind it, and any policy gates — a reviewable account of intent before action.
5. **Render**
   synthesis, sampling, performance, mixing, space, and mastering run as deterministic computation, every random path seeded from the contract hash.
6. **Deliver & verify**
   the output is a WAV plus a manifest of hashes, loudness, and markers; verification re-renders and compares bytes — the gate against stale or drifted results.

V3

The pipeline, end to end

How a source becomes a verified, reproducible asset



Compilation fixes intent as a hashed contract; validation proves it render-ready; the plan records every routing decision; the render is fully seeded; the manifest carries provenance; verify closes the loop.

6 SECTION Key capabilities

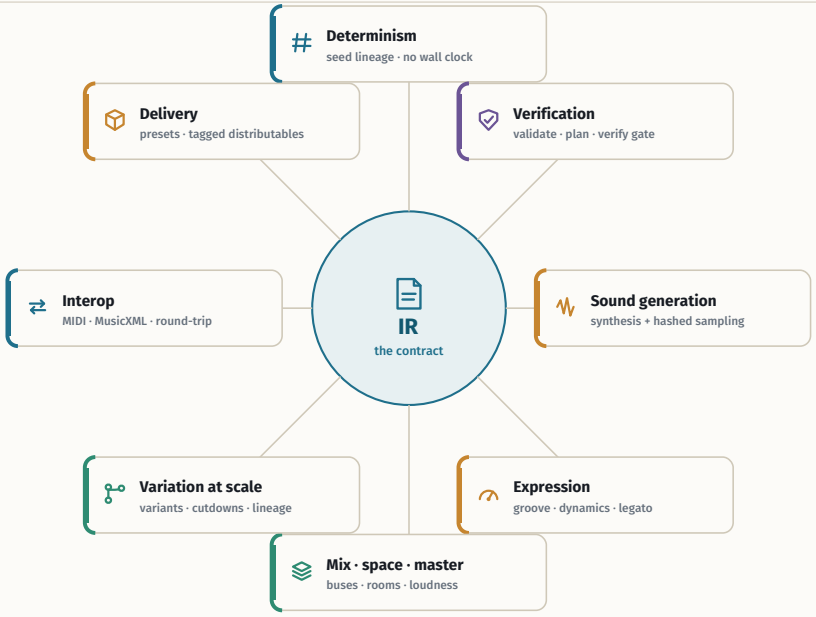
The capabilities are not a feature list bolted together; they compose because they all read from and write to the same contract. Each resolves into the explicit IR, which is exactly why combining them produces no surprises.

- The contract — macros, emotional targets, motifs, and style resolve at compile time into explicit parameters, so the IR always says exactly what will render.
- Determinism — one seed lineage rooted in the contract hash makes noise, dither, round-robins, and reverb tails reproducible.
- Verification — validation-as-render-ready, the render plan, and the byte-compare gate, with policy controls that warn or refuse at a boundary.
- Sound generation — seeded synthesis alongside a content-addressed sampler that pins instrument libraries by hash.
- Expression — groove, humanization, dynamics-as-timbre, legato, and section ensembles, all expanded into inspectable values.
- Mix, space, and master — buses with shared processing, sidechain control, equalization, loudness targeting, and physically-motivated rooms.
- Variation at scale — seeded variants, structural choice sets, campaign cutdowns, and intensity ladders, each hash-tracked with lineage.
- Interoperation and delivery — standard interchange in and out, a round-trip back to source, and presets that package tagged distributables.

V4

Capabilities compose around one contract

Every capability reads from and writes to the same inspectable IR



Nothing acts on private state. Generation, expression, mixing, variation, interop and delivery all resolve into the same explicit IR — which is exactly why they compose without surprises.

7 SECTION Experience for AI agents

For an agent, AudioForge is an environment designed to be perceived, acted upon, and trusted without a human in the loop.

Perceive

The agent reads the IR directly and consults a capability manifest that enumerates the available primitives and their parameters. Diagnostics are structured and cite exact source locations; the render plan exposes every routing decision and its evidence; the manifest can carry analysis features and recommendations that feed a critique loop.

Act

The agent authors the source or emits the IR outright, and mutates it through schema-checked edits. Authoring never raises on imperfect input — every issue returns as a structured diagnostic — so the agent iterates against feedback instead of exceptions.

Verify and benefit

Because model output is treated as untrusted by default, the agent's work passes through the same validation and byte-compare gate as anyone's. The payoff is a closed loop — propose, inspect, verify, repair — a way to produce audio that is safe to drop into a pipeline precisely because its correctness is checked, not assumed.

8 SECTION Experience for human operators

For a person, the system trades the reassurance of a familiar interface for something more durable: control that leaves a record.

Direct

Operators write austere, high-level intent — emotional targets, intensity shapes, motifs, style — and watch it expand into explicit, reviewable parameters. Nothing meaningful happens off the page.

Oversee

Manifests, markers, and loudness reports make each render legible; two renders can be diffed; the verify gate guards deliveries in continuous integration. Oversight is reading and comparing artifacts, not trusting a memory.

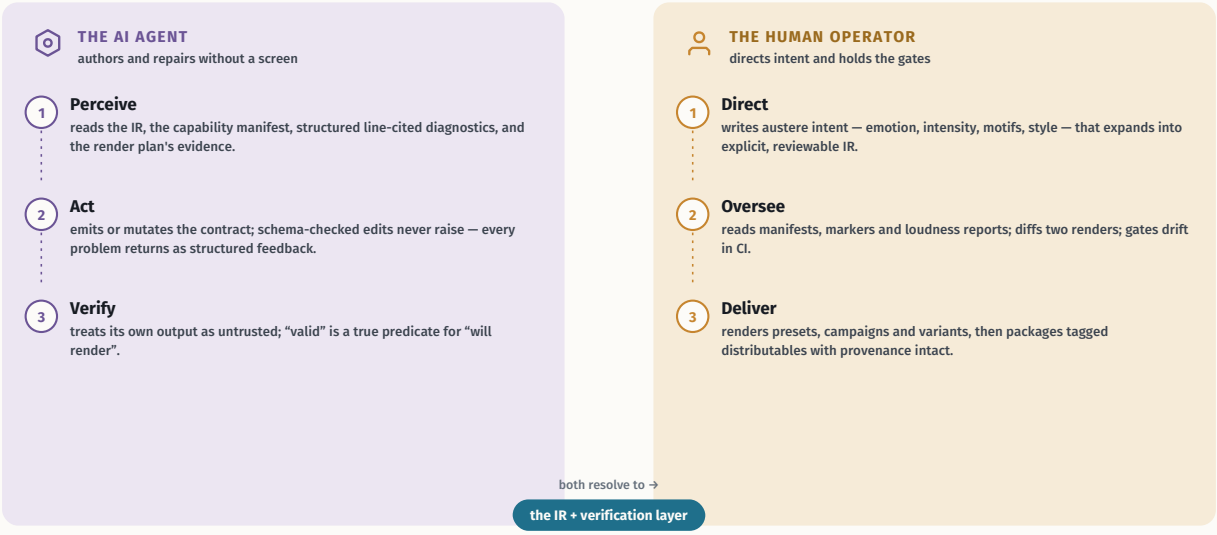
Work with

A software interface and command surface cover validation, compilation, rendering, batching, campaigns, and verification, alongside preview and delivery presets. The operator's role shifts from performing a session to directing and reviewing a build.

V5

Two hands on the same contract

How an AI agent and a human operator each work the system



Both meet at the IR and its verification layer. The agent reads and writes the contract directly; the operator shapes intent and holds the gates — neither depends on a UI to absorb ambiguity.

9 SECTION Design principles

A handful of beliefs decide the tradeoffs, and they are ranked so conflicts resolve predictably — reproducibility and loud failure outrank convenience; explicit data outranks clever implicit behavior.

- The IR is truth — authoring is convenience; the contract is the data that renders.
- Reproducibility is a product feature — an output is not finished until it can be recreated and verified.
- Declared variation beats accidental drift — every stochastic element is seeded and traceable.
- Failure is loud and local — missing resources, ambiguity, and unsupported requests produce line-cited errors, never a silent swap.
- Model output is untrusted by default — verification is a first-class design concern, on par with input validation.
- Explicit over implicit — every choice that affects the result is visible in the contract.
- Determinism is checked, not assumed — cross-environment identity is proven by a gate.
- Scope is a decision — non-vocal by design, depth of craft over breadth of surface.

V6

The beliefs the system is built on

Eight principles that decide every tradeoff

The IR is truth 01
authoring is sugar; the contract is the data that renders.

Reproducibility is a feature 02
byte-identical, or the asset isn't done.

Declared over accidental 03
variation is seeded and traceable, never leaked.

Fail loud and local 04
missing or unsupported → line-cited errors, never silent swaps.

LLM output is untrusted 05
verification is a first-class design concern.

Explicit over implicit 06
every render-affecting choice is visible in the IR.

Determinism is checked 07
cross-environment identity is proven by a gate, not assumed.

Scope is a choice 08
non-vocal by design — depth over surface area.

When two options conflict, these rank the decision. Reproducibility and loud failure outrank convenience; explicit data outranks clever implicit behavior.

SECTION 10

Implementation direction

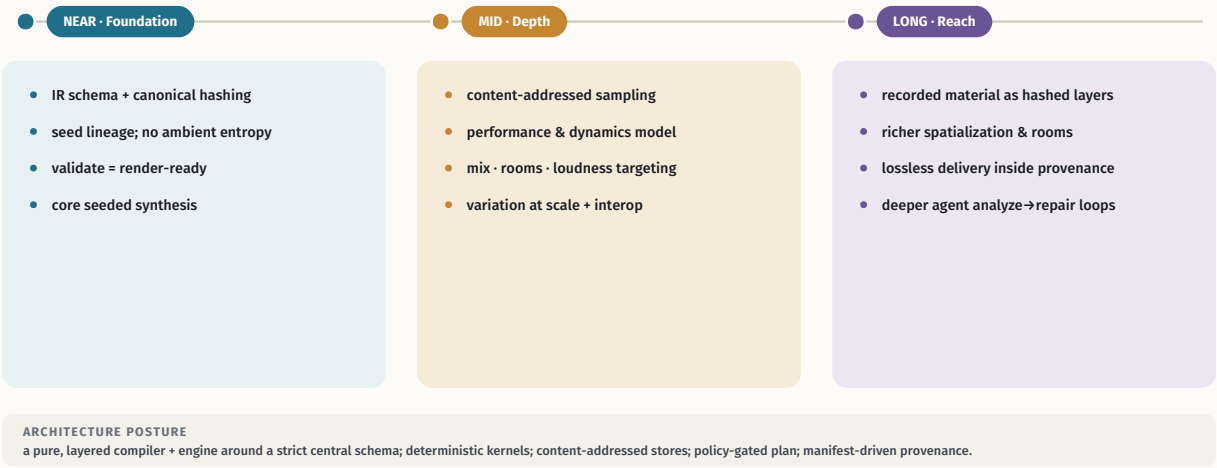
The architecture posture is a pure, layered compiler and engine organized around a strict central schema. Authoring surfaces sit at the top; a canonical, hashed IR sits at the center; deterministic signal-processing kernels sit below, with any that require native compilation failing loudly rather than substituting different audio. Resources — samples, instrument libraries, impulse responses — live in content-addressed stores whose hashes participate in the asset's identity. A policy-gated render plan sits between validation and rendering; a manifest carries provenance out the other side.

Sequencing follows dependency, not appetite. The contract, the determinism guarantee, and render-ready validation come first, because everything later leans on them. Sampling and libraries, the performance and dynamics model, mixing, rooms, loudness, variation at scale, and interoperation build on that base. Broader palette, richer spatialization, lossless delivery inside a provenance envelope, and deeper agent loops come last. Each phase ships as a whole and stays reproducible; nothing is postponed into an unverifiable future.

V7

A path that stays verifiable at every step

Near-, mid- and long-term direction over a single spine



ARCHITECTURE POSTURE
a pure, layered compiler + engine around a strict central schema; deterministic kernels; content-addressed stores; policy-gated plan; manifest-driven provenance.

The contract, determinism and the render-ready gate come first because everything later leans on them. Each phase ships whole and stays reproducible; nothing is deferred into an unverifiable future.

11 SECTION Risks, constraints, tradeoffs

The boundaries are stated plainly, because hiding them would violate the system's own stance.

- Cross-platform floating-point identity can vary by environment — proven per environment with the verify gate rather than assumed universally.
- Loudness measurement and limiting are production-consistent approximations, not certified metering — manifests record measured values for checking against compliant meters.
- Sampled sustains are finite; a held note can outrun a non-looping sample — the render plan detects the gap and, by policy, warns or refuses rather than emitting unintended silence.
- Some voices and effects depend on native kernels that need a compiler — they fail loudly rather than silently substituting different audio.
- Lossy delivery formats sit outside the byte-hash guarantee — provenance is carried in the delivered file's metadata instead.
- The non-vocal scope is a deliberate constraint — it concentrates craft rather than spreading it thin.

> *Every limit is met the same way: make the boundary explicit and auditable, rather than papering over it.*

V8

Honest boundaries, and how each is held

The constraints that shape the system — and the deliberate response to each

CONSTRAINT	THE LIMIT	HOW IT'S HELD
Cross-platform float identity	reproducibility can vary by environment	the verify gate proves identity per environment, rather than assuming it
Loudness & limiting	production-consistent, not a certified meter	manifests record measured values for reference against compliant meters
Finite sampled sustains	held notes can outrun a non-looping sample	the plan detects the gap and warns or refuses, per policy
Native DSP kernels	some voices/effects need a compiler	they fail loud — never a silent substitution of different audio
Lossy delivery (MP3)	sits outside the byte-hash guarantee	provenance is carried in the file's metadata instead
Non-vocal scope	voice and lyrics are out of scope	a deliberate boundary — depth of craft over breadth of surface

None of these are hidden. Each limit is met by making the boundary explicit and auditable rather than papering over it — the same loud-failure stance the whole system is built on.

12 SECTION Evolution paths

The concept has room to grow without loosening its guarantees.

- Recorded material enters cleanly as content-addressed sample layers, bringing field recordings and live takes under the same hash discipline.
- Spatialization deepens toward full-sphere formats and a wider library of measured rooms.
- Delivery gains lossless, encoder-independent paths wrapped in a provenance envelope, extending trust beyond the raw byte hash.
- Agent loops grow richer — analysis, targeted repair, and multi-asset campaigns — turning the critique cycle into a standing capability.
- A shared capability and interchange vocabulary lets independent tools and agents cooperate around the same contract.

> ***In every direction the invariant holds: whatever is added must remain inspectable, seeded, and verifiable. That constraint is not a limit on the idea — it is the idea.***